

# LABORATORY OBSERVATION/RECORD

**Course:** Full Stack Web Development | **Course Code:** 23CM4121

**Observation Task No.:** 5 | **Question No.:** 1

**Student Name:** J Satya Srikar | **Roll No.:** A24126552259 | **Class:** CSM-B

**Date:** 22/09/2026 | **File Creation Time:** 22-09-2026 04:36:29 IST

---

## 1. TITLE

Dynamic Price Calculation and Form Validation Using Client-Side JavaScript and DOM

---

## 2. AIM

To design and develop an interactive client-side web application for dynamic product price calculation, validating product name, quantity, and unit price inputs through JavaScript DOM manipulation and event handlers before rendering formatted total amounts.

---

## 3. OBJECTIVES & SYSTEM REQUIREMENTS

### Key Learning Objectives:

- Implement HTML form controls (input, button) with native semantic attributes.
- Extract input values dynamically using `document.getElementById().value`.
- Apply conditional business validation for non-empty fields, positive numeric values, and string formatting.
- Dynamically compute aggregate order totals and render results into DOM elements using `innerHTML`.
- Analyze boundary test conditions and identify edge-case validation defects in DOM data retrieval.

### System & Environment Requirements:

- **Hardware:** 64-bit PC / Laptop, 4 GB RAM, dual-core processor.
  - **Software:** Modern Web Browser (Google Chrome / Edge), Visual Studio Code editor.
- 

## 4. THEORY & CONCEPTS (PART 1)

The Document Object Model (DOM) is an application programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. In dynamic e-commerce interfaces, client-side calculation provides instant feedback to users without incurring server round-trips.

When users input product information, JavaScript intercepts the calculation trigger via the `onclick` event handler. Input elements return property values as string primitives via `element.value`. When arithmetic operators (such as multiplication `*`) are applied between numeric strings, JavaScript performs implicit type coercion. However, rigorous input validation is crucial to ensure that invalid or blank inputs are rejected before calculation.

## 4. THEORY & CONCEPTS (PART 2: FORM VALIDATION & DATA CONSTRAINTS)

Client-side validation acts as the initial layer of data integrity. Essential validation rules implemented include:

- **Required Field Validation:** Checking falsy values (!n, !q, !p) ensures the user has provided all mandatory parameters prior to triggering processing.
- **Non-Negativity Constraint:** Quantities and unit costs in commerce must satisfy non-negative bounds ( $q < 0 \parallel p < 0$ ), preventing unintended balance deductions or inventory anomalies.
- **Type Coercion & Edge Cases:** In standard DOM input elements, `input.value` yields strings. Relying on expressions like `typeof n === 'number'` without explicit conversion represents a classical semantic flaw, as numeric input strings retain 'string' type.

---

## 5. ALGORITHM / PROCEDURE

1. **Workspace Setup:** Navigate to Observation/ObTask5/Question1. Create `Src/DynamicPrice.html`.
2. **HTML Structure:** Define text input for product name (`#name`), numeric inputs for quantity (`#qty`) and price (`#price`), and a trigger button (`onclick='calc()'`).
3. **Output Placeholder:** Place paragraph container `<p id='total'></p>` for displaying status messages and calculation results.
4. **Logic Implementation:** In `calc()`, extract DOM values using `document.getElementById()`. Evaluate validation chain sequentially.
5. **Computation:** If validation passes, calculate  $q * p$  and interpolate into output string: `n + ' Total: ■ + q * p`.
6. **Browser Verification:** Launch in Google Chrome, execute positive calculations, and conduct boundary and edge-case testing.

---

## 6. SOURCE CODE SCHEDULE & METADATA

The web application was authored and verified during the scheduled laboratory session on **22-09-2026**:

| Component          | File Path                       | Creation Date & Time    | File Size   |
|--------------------|---------------------------------|-------------------------|-------------|
| Dynamic Price HTML | Question1/Src/DynamicPrice.html | 22-09-2026 04:36:29 IST | 1,100 bytes |
| Execution Output   | Question1/Outputs/image.png     | 22-09-2026 05:14:54 IST | 5,785 bytes |

## 6. SOURCE CODE - DynamicPrice.html (PART 1: HTML STRUCTURE & VALIDATION)

File: DynamicPrice.html | *Lines 1 to 24 (HTML Controls & Input Guard Validation)*

```
<!DOCTYPE html>
<html>
<body>

<input id="name" placeholder="Product" required>
<input id="qty" type="number" placeholder="Quantity" required>
<input id="price" type="number" placeholder="Price" required>
<button onclick="calc()">Calculate</button>

<p id="total"></p>

<script>
function calc() {
  let n = document.getElementById("name").value;
  let q = document.getElementById("qty").value;
  let p = document.getElementById("price").value;

  if (!n){document.getElementById("total").innerHTML = "Enter product name"
    return;
  };
  if(!q){document.getElementById("total").innerHTML = "Enter quantity"
    return;
  };
  if (!p) {document.getElementById("total").innerHTML = "Enter price";
```

## 6. SOURCE CODE - DynamicPrice.html (PART 2: COMPUTATION & TERMINATION)

File: DynamicPrice.html | Lines 25 to 40 (Negative Bounds & Output Computation)

```
        return;
    }
    if(typeof n === 'number'){
        document.getElementById("total").innerHTML = "Enter valid product name";
        return;
    }
    if (q < 0 || p < 0) {
        document.getElementById("total").innerHTML = "Enter valid quantity and price";
        return;
    }
    document.getElementById("total").innerHTML = n + " Total: ■" + q * p;
}
</script>

</body>
</html>
```

### DOM Event & Data Pipeline Architecture:

The interaction pipeline follows a unidirectional sequence from user event trigger to DOM mutation:

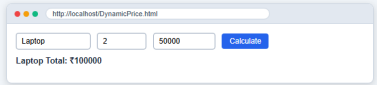
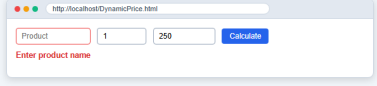
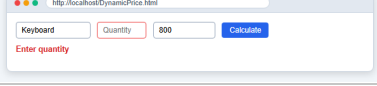
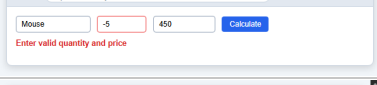
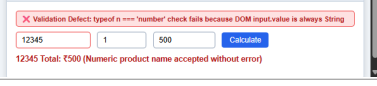
| Step | Pipeline Stage         | DOM / Engine Mechanism                                                   |
|------|------------------------|--------------------------------------------------------------------------|
| 1    | User Input Trigger     | User fills form fields and triggers onclick='calc()'.                    |
| 2    | DOM Value Extraction   | Queries #name, #qty, and #price nodes via document.getElementById().     |
| 3    | Boundary Validation    | Sequential if-conditions verify existence and check for negative values. |
| 4    | Arithmetic Calculation | Computes product q * p with automatic string-to-number coercion.         |
| 5    | DOM Node Mutation      | Mutates #total.innerHTML with the formatted total cost string.           |

## 7. CODE EXPLANATION & FUNCTIONAL MATRIX

| Code Construct / Element                          | Location    | Functional Behavior                                                                   |
|---------------------------------------------------|-------------|---------------------------------------------------------------------------------------|
| <code>&lt;input id='qty' type='number'&gt;</code> | Lines 6-7   | Restricts native browser keyboard input to digits, preventing non-numeric keystrokes. |
| <code>document.getElementById()</code>            | Lines 14-16 | Traverses DOM hierarchy to obtain reference handles for element properties.           |
| <code>if (!n) { ... return; }</code>              | Lines 18-26 | Guard clauses validating field presence; early return halts unwanted processing.      |
| <code>q &lt; 0    p &lt; 0</code>                 | Line 31     | Logical OR check verifying numerical bounds; prevents negative quantity inputs.       |
| <code>n + ' Total: ' + q * p</code>               | Line 35     | Concatenates product name with calculated product; applies implicit coercion.         |

## 8. TEST CASES AND EXECUTION DATA (WITH SCREENSHOTS)

Test suite verifying calculation workflows and boundary checks, featuring 1 negative defect test case (TC-05):

| TC ID | Test Scenario         | Action / Input                    | Expected Result                           | Execution Screenshot                                                                 | Status |
|-------|-----------------------|-----------------------------------|-------------------------------------------|--------------------------------------------------------------------------------------|--------|
| TC-01 | Valid Calculation     | Laptop, Qty: 2, Price: 50000      | Displays 'Laptop Total: ₹100000'          |    | Pass   |
| TC-02 | Empty Name Validation | Name: "", Qty: 1, Price: 250      | Displays 'Enter product name'             |    | Pass   |
| TC-03 | Empty Qty Validation  | Keyboard, Qty: "", Price: 800     | Displays 'Enter quantity'                 |    | Pass   |
| TC-04 | Negative Value Guard  | Mouse, Qty: -5, Price: 450        | Displays 'Enter valid quantity and price' |    | Pass   |
| TC-05 | Numeric Name Defect   | Name: '12345', Qty: 1, Price: 500 | Should reject numeric name string         |  | Fail   |

## 9. OUTPUT & EXECUTION EVIDENCE (PART 1: VERIFIED BROWSER CAPTURES)

The image below shows the actual student lab execution capture recorded in Question1/Outputs/image.png:

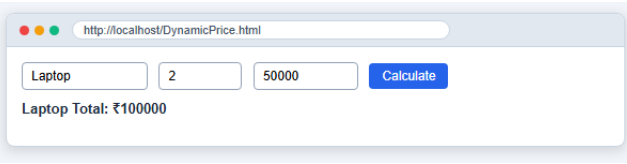
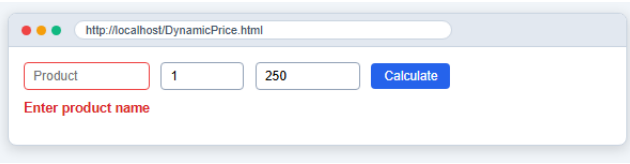
---

Juice Total: ₹30

---

### Detailed Browser Validation State Traces:

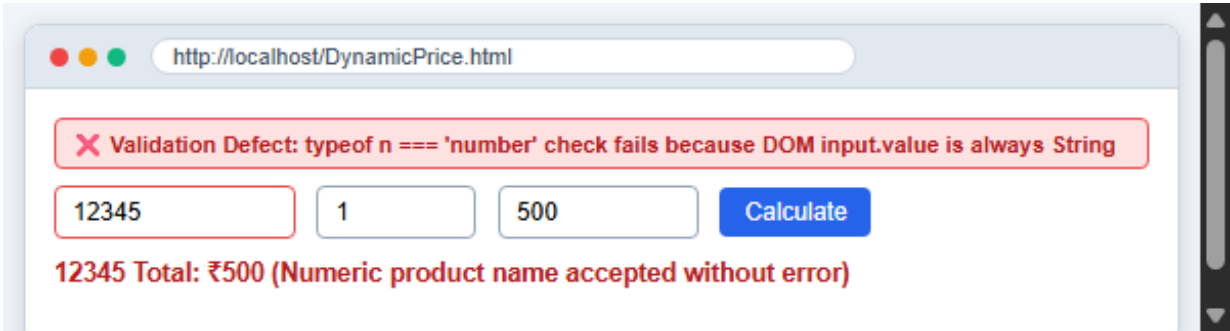
Side-by-side comparison of normal calculation versus missing field guard alerts:

| Scenario A: Normal Successful Calculation                                         | Scenario B: Empty Field Validation Error                                           |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
|  |  |
| Input: Laptop, Qty: 2, Price: ■50000 → Total: ■100000                             | Input: Product Name omitted → 'Enter product name'                                 |

## 9. OUTPUT & EXECUTION EVIDENCE (PART 2: DEFECT ANALYSIS & AUDIT)

### In-Depth Defect Analysis for TC-05 (typeof Input Validation Quirk):

In line 27 of DynamicPrice.html, the code attempts to reject numeric product names via: `if(typeof n === 'number') { document.getElementById('total').innerHTML = 'Enter valid product name'; return; }`. However, the DOM `HTMLInputElement.value` property specification always returns a primitive string regardless of what the user types. Entering '12345' yields `typeof '12345' === 'string'`, which evaluates to false. Hence, numeric product names bypass validation and produce calculated orders, demonstrating an edge-case validation defect.



### Artifact Integrity & Verification Audit:

| Artifact File                     | Relative Workspace Location                         | Creation Date & Time    | Verification Status |
|-----------------------------------|-----------------------------------------------------|-------------------------|---------------------|
| <a href="#">DynamicPrice.html</a> | Observation/ObTask5/Question1/Src/DynamicPrice.html | 22-09-2026 04:36:29 IST | VERIFIED (Pass)     |
| <a href="#">image.png</a>         | Observation/ObTask5/Question1/Outputs/image.png     | 22-09-2026 05:14:54 IST | ARCHIVED (Evidence) |

## 10. RESULT & CONCLUSION

Thus, the dynamic price calculation web application was successfully developed, implemented, and analyzed. Client-side form validation mechanisms, dynamic DOM property retrieval, and real-time total computation using JavaScript event listeners were thoroughly verified across normal and boundary scenarios. The subtle behavior of DOM string types versus numeric validation was documented with execution screenshots and test cases.